

Clearance Level System

The StockWise application enforces a strict hierarchical clearance model to ensure system integrity and operational security. By implementing the **Principle of Least Privilege**, we restrict user access to only the resources necessary for their specific job functions, managed centrally through our server-side security configuration.

4.1 Hierarchical Role Definitions

Our system defines two primary clearance tiers, each mapped to a set of authorities within the Spring Security framework.

- **Administrative Level (ROLE_ADMIN):**
 - **Full System Governance:** Admins possess unrestricted access to the entire system infrastructure.
 - **User Lifecycle Management:** Capability to create, update, deactivate, or delete user accounts.
 - **System Configuration:** Ability to modify global settings, manage master data, and configure business rules.
 - **Strategic Reporting:** Access to comprehensive, cross-organizational business intelligence reports and audit trails.
- **Operational Staff Level (ROLE_STAFF):**
 - **Functional Inventory Management:** Restricted access focused exclusively on operational tasks, such as updating current stock counts and logging transactions.
 - **Query & Reporting:** Permission to view product details and generate basic, operational-level reports.
 - **Constraint-Based Access:** Staff members are prohibited from altering system-wide configurations, managing other users, or accessing sensitive financial data not relevant to their daily operations.

4.2 Enforcement Strategy & Security Posture

To ensure clearance levels are strictly honored in our monolithic architecture, we enforce authorization checks at the Controller and View layers, utilizing Spring Security's **Method-Level Security** and **Thymeleaf Security Dialect**.

- **Declarative Security:** All Controller methods are protected by declarative security annotations (e.g., `@PreAuthorize("hasRole('ADMIN')")`). This ensures that authorization logic is co-located with the business logic, preventing "authorization bypass" vulnerabilities before the view is rendered.
- **Default Deny Policy:** The system follows a "secure-by-default" approach. Any URL path or Controller action not explicitly annotated or configured with access permissions is automatically blocked, ensuring new features are secure by design.
- **View-Level Protection:** By utilizing the Thymeleaf Security Dialect (e.g., `sec:authorize="hasRole('ADMIN')"`), we conditionally render UI components. Sensitive buttons (e.g., "Delete User") are not even generated in the HTML for users lacking the necessary clearance, preventing unauthorized users from even seeing restricted actions.

- **HTTP Status Handling:** When an unauthorized request is attempted directly (e.g., via URL manipulation), the system immediately terminates the request and returns an **HTTP 403 Forbidden** status code, effectively blocking access to protected pages or resources.

4.3 Auditability and Compliance

To further enhance the robustness of our clearance system, we implement an **Audit Logging** layer.

- **Traceability:** Every administrative action performed by an `ADMIN` (such as changing user permissions or deleting inventory logs) is recorded in an audit database. This provides a forensic trail, allowing for accountability and compliance monitoring, which is an industry standard for professional inventory systems.
- **Temporal Context:** Each action is logged with a timestamp, the unique identifier of the user who performed the action, and the outcome of the request, ensuring transparency across all clearance levels.